

a large-area target and has a long-lasting effect. For example, if an attacker steals a laptop or a backup of a database, they have access to all the records accumulated over several years. Threats include physical theft of devices, decommissioned drives that were never wiped, insider access to storage systems, and attackers who gain a foothold on a server and read files directly. Encryption at rest ensures that the raw bytes on the storage medium are ciphertext, so possession of the medium alone yields nothing.

Data in transit: Data in transit is information moving between systems: a browser talking to a web server, microservices exchanging API calls, database replication streams, file transfers, and emails. While travelling, data traverses networks the sender does not control — public Wi-Fi, internet backbones, cloud provider infrastructure where it can be passively sniffed or actively intercepted through man-in-the-middle attacks, ARP spoofing, or rogue access points. Encryption in transit wraps the communication in a secure channel, typically using the Transport Layer Security (TLS) protocol, guaranteeing confidentiality (no eavesdropping), integrity (no

tampering), and authentication (you are talking to the genuine server).

A complete data protection strategy must address both states: perfectly encrypted disks do not help if credentials cross the network in plaintext, and flawless TLS does not help if the database behind it stores card numbers unencrypted.

Encrypting data in transit with open source tools

OpenSSL and TLS 1.3: OpenSSL is the foundational open source cryptography library that implements the TLS protocol along with a comprehensive toolkit of ciphers, hashes, and certificate utilities. Web servers such as Nginx and Apache, mail servers, and countless applications link with it to provide encrypted channels. The current best practice is to enable TLS 1.3, which removes the legacy weak ciphers of earlier versions, encrypts more of the handshake, and provides forward secrecy by default. This means that even if a server's private key is compromised later, previously recorded traffic cannot be decrypted. Administrators use the OpenSSL command-line tool to generate private keys, create certificate

signing requests, inspect certificates, and test server configurations.

Let's Encrypt and Certbot: A TLS certificate proves a server's identity, and until 2015 obtaining one meant paying a commercial certificate authority. Let's Encrypt changed this by offering free, automatically issued certificates through the ACME protocol. Its companion client, Certbot, handles the entire lifecycle: it proves domain ownership, installs the certificate into the web server configuration, and renews it automatically before its 90-day expiry. This automation removed the most common excuse for running unencrypted websites, and HTTPS adoption on the web has climbed above 90 percent largely as a result. For internal services, the same ACME workflow can be replicated privately with the open source step-ca certificate authority.

VPNs — WireGuard and OpenVPN: Not all traffic is web traffic. Site-to-site links, remote administration, and access to internal networks are best protected with a virtual private network. WireGuard is an open source implementation that uses about 4000 lines of code in the kernel; it implements ChaCha20-Poly1305

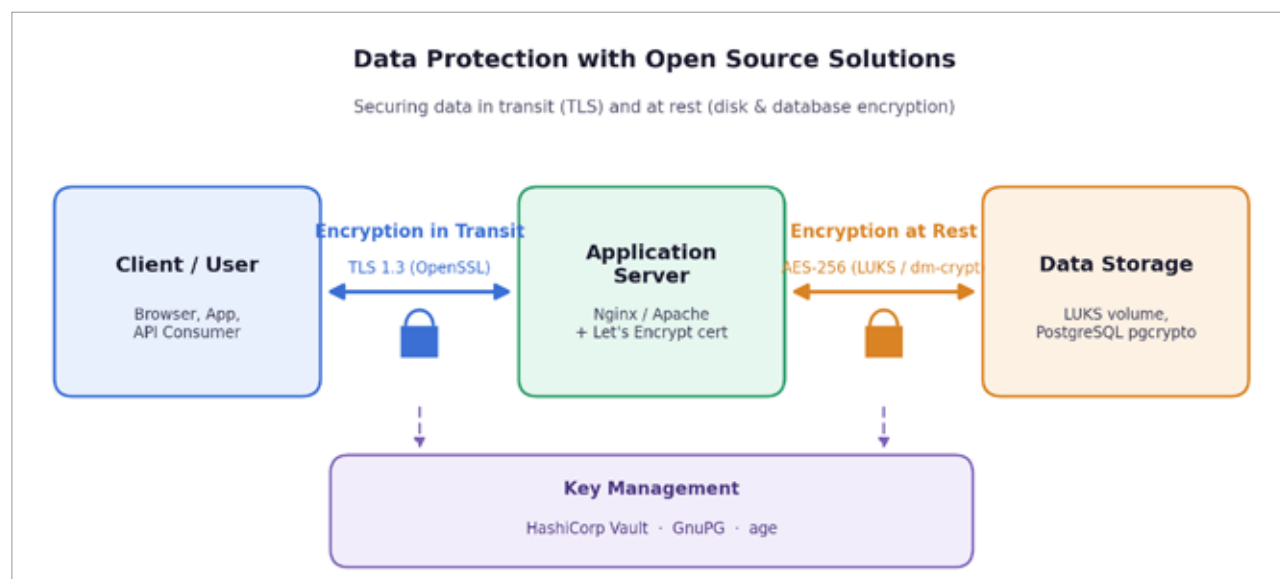


Figure 1: Open source encryption applied across the data lifecycle